1. 왜 AOP 를 쓰는가 - 보완

> 본 글은 **1. 왜 AOP를 쓰는가**에 이어, AOP를 이해하는 데 필요한 핵심 용어를 정리한 글이다.

AOP 문서를 처음 읽으면 Join point, Pointcut, Advice, Aspect 같은 용어가 한꺼번에 쏟아져서 헷갈리기 쉽다. 이 글에서는 **"모든 음식의 제조 시간을 측정한다"** 는 하나의 예제를 따라가면서 각 용어가 어떤 역할을 맡는지 순서대로 정리한다.

Join point : 부가 코드를 끼울 수 있는 실행 지점

Join point는 프로그램이 실행되는 동안 발생하는 특정 순간이다. 이론적으로는 **메소드 호출, 필드 읽기·쓰기, 생성자 실행, 예외 처리 등**이 모두 Join point가 될 수 있다. AspectJ 같은 완전한 AOP 프레임워크는 실제로 이 지점들을 대부분 지원한다.

하지만 Spring AOP에서 Join point는 **"Spring Bean의 메소드가 실행되는 순간"** 하나뿐이며, 필드 접근이나 생성자 실행은 가로챌 수 없다. 코드에서는 Advice 메소드의 파라미터로 받는 `JoinPoint`, `ProceedingJoinPoint`객체가 바로 이 실행 지점을 표현한 것이다.

> Join point는 AOP가 끼어들 수 있는 지점이다. 즉, **"적용 대상이 될 수 있는 후보"** 라고 생각하면 편하다.

```
makeCookie()
makeBread()
```

```
makeCake()
makeDrink()
```

위 네 개의 메소드 실행 자체가 각각 Join point다.

여기서 **"모든 음식의 제조 시간을 측정하고 싶다"** 는 요구사항이 생겼다고 해보자. 이때 **공통 관심사(Cross-cutting Concern)** 는 음식의 제조 시간 측정이고, Join point는 위 메소드들의 실행 지점이다.

예제를 통해 다음과 같이 **Join point**를 이해할 수 있다.

- Spring Bean의 모든 메소드 실행이 Join point의 후보가 될 수 있다.
- 위 메소드들이 모두 Spring Bean의 메소드라면 각각의 실행 지점은 Join point가 될 수 있다.
- 반대로 `new`로 직접 생성한 객체처럼 Spring이 관리하지 않는 객체의 메소드는 Join point가 될 수 없다.

> **참고 : Spring AOP의 Join point 제약** Spring AOP는 프록시 기반으로 동작하기 때문에 다음과 같은 경우에는 Advice가 적용되지 않는다.
>
> - `private`, `final`, `static` 메소드 (프록시가 오버라이드할 수 없다)
> - 같은 객체 내부에서 `this.method()` 형태로 호출하는 경우 (자기 호출, Self-invocation)
>
> 자세한 이유는 아래 **AOP Proxy** 부분에서 다시 다룬다.

Pointcut : 어떤 Join point를 대상으로 할지 고르는 조건식

모든 메소드 실행에 부가 코드를 넣을 수는 없으니 **대상을 골라야 한다.** 그 조건이 **Pointcut**이다.

```
@Pointcut("execution(* com.example.service..*(..))")
public void serviceLayer() {} // 본문은 비워둔다. 메소드 이름이 곧 Pointcut의 이름이 된다.
```

> `com.example.service` 패키지와 그 하위 패키지에 있는 모든 메소드의 실행을 대상으로 하는 조건이다.

Spring은 각 메소드 실행(Join point)에 대해 이 조건이 참인지 거짓인지를 판정한다. 공식 문서가 Pointcut을 **predicate(참/거짓을 판정하는 조건식)** 라고 부르는 이유다. 이름이 아니라 조건으로 대상을 고르기 때문에, 나중에 이 패키지에 새 메소드가 추가돼도 별도 설정 없이 자동으로 대상이 된다.

`@Pointcut`을 붙인 메소드는 반환 타입이 `void`여야 하며, 본문은 실행되지 않으므로 의미가 없다. 이렇게 정의해두면 Advice에서 `@Around("serviceLayer()")`처럼 이름으로 재사용할 수 있다.

```
makeCookie(), makeBread(), makeCake(), login(), save(), delete()
→ 각 메소드의 실행 지점은 모두 Join point가 될 수 있다.

하지만 Pointcut을 execution(* com.example.food..*.make*(..)) 로 작성하면
makeCookie(), makeBread(), makeCake() 등은 선택되고
login(), save(), delete() 등은 제외된다.
```

즉, 여러 후보(Join point)들 중에서 실제 대상을 추출하는 조건이 Pointcut이다.

execution 표현식 뜯어보기

```
execution(
    *                       → 반환 타입
    com.example.food..*     → 패키지 + 클래스(타입)
    .make*                  → 메소드 이름
    (..)                    → 파라미터
)
```

`execution(..)` : 메소드 실행을 기준으로 대상을 잡겠다는 Pointcut 지시자(Designator)다.

- **맨 앞의** `*` : 반환 타입이 무엇이든 상관없다는 뜻이다. : 반환 타입을 제한하고 싶으면 `*` 자리에 타입을 직접 쓴다. 예를 들어 `execution(String com.example.food..*.make*(..))`라고 쓰면 `String`을 반환하는 메소드만 대상이 된다.
- **`com.example.food..*`** : 어떤 클래스(타입)인지 지정하는 부분이다. : 여기서 `..`는 해당 패키지뿐 아니라 하위 패키지까지 포함한다는 뜻이다. : 마지막 `*`는 클래스 이름이 무엇이든 가능하다는 뜻이다. : 즉, `com.example.food` 패키지와 그 하위 패키지에 존재하는 모든 타입이다. : 참고로 `com.example.food.*`처럼 `.`을 하나만 쓰면 하위 패키지는 포함하지 않는다.
- **`.make*`** : 메소드 이름이다. : 즉, `make`로 시작하는 모든 메소드다.
- **파라미터 패턴**

패턴	의미
`(..)`	파라미터가 몇 개든, 타입이 무엇이든 상관없다 (0개 포함)
`()`	파라미터가 없는 메소드
`(*)`	파라미터가 정확히 1개이며 타입은 아무거나 가능하다
`(String)`	`String` 하나만 가능하다

패턴	의미
`(String, *)`	첫 번째는 `String`, 두 번째는 아무 타입 하나 (정확히 2개)
`(String, ..)`	첫 번째는 `String`, 그 뒤로는 개수·타입 상관없다

다시 읽어보면 **`com.example.food` 및 하위 패키지에 있는 모든 클래스 중에서, 반환 타입과 파라미터에 상관없이 `make`로 시작하는 메소드의 실행을 선택한다**는 뜻이다.

execution 외의 지시자

`execution`이 가장 많이 쓰이지만, 상황에 따라 다른 지시자도 자주 사용한다.

지시자	의미	예시
`within`	특정 타입(패키지) 안의 모든 메소드	`within(com.example.food..*)`
`@annotation`	특정 어노테이션이 붙은 메소드	`@annotation(com.example.annotation.Timer)`
`bean`	특정 이름의 Spring Bean (Spring AOP 전용)	`bean(*Service)`

여러 조건은 `&&`, `||`, `!`로 조합할 수도 있다.

Advice : 선택된 Join point에서 실행할 부가 로직

- `Join point` : AOP가 개입할 수 있는 메소드 실행 지점
- `Pointcut` : 그 Join point들 중 실제로 적용할 대상을 고르는 조건

Advice는 Pointcut으로 선택된 Join point에서 실제로 실행할 부가 로직이다. 지금 예제의 공통 관심사는 각 음식의 조리 시간을 측정하는 것이다. Pointcut은 적용 대상을 고르는 것까지만 할 뿐, 아직 실제로는 아무것도 하지 않는다. 그래서 무엇을 할지 로직을 작성해야 하는데, 그것이 Advice다.

```
@Pointcut("execution(* com.example.food..*.make*(..))")
public void makeFood() {}

@Around("makeFood()")
public Object measureTime(ProceedingJoinPoint
joinPoint) throws Throwable {

    long start = System.currentTimeMillis();
    Object result = joinPoint.proceed(); // 원본 메소드
(makeCookie 등) 실행
    long end = System.currentTimeMillis();

    // 생략

    return result;
}
```

여기서 `measureTime()` 메소드가 Advice이다. 즉, Pointcut에 선택된 메소드에 대해 Advice로 **무엇을 할지** 정의해두는 것이다. 그리고 단순히 무엇을 할지뿐 아니라 **언제 할지**도 Advice에 포함된다.

```
@Before : 실행 전

makeCookie()
- 정상적으로 반환       : @AfterReturning
- 예외 발생             : @AfterThrowing
- 성공·실패 상관없이    : @After

> 이 전체를 앞뒤로 감싸면서 직접 제어하는 것이 @Around이다.
```

- `@Before` : 원본 메소드가 실행되기 직전에 Advice가 실행된다. : 예외를 던지지 않는 한 원본 메소드의 실행을 막을 수 없다.

- `@AfterReturning` : 원본 메소드가 정상적으로 끝난 뒤 실행된다. : 예외가 발생하면 실행되지 않는다. : `returning` 속성으로 반환값을 받아볼 수 있다.
- `@AfterThrowing` : 원본 메소드에서 예외가 발생하면 실행된다.
  : `throwing` 속성으로 발생한 예외를 받아볼 수 있다.
- `@After` : `finally`와 비슷하게, 정상 종료든 예외 발생이든 항상 실행된다.
- `@Around` : 원본 메소드를 통째로 감싼다. : `joinPoint.proceed()`를 호출해야 원본 메소드가 실행되며, 호출하지 않으면 원본 메소드가 아예 실행되지 않는다. : 전달 인자나 반환값을 바꾸거나, 발생한 예외를 다른 예외로 바꾸는 것도 가능하다.

> `@Around`가 가장 강력하지만, 공식 문서는 **필요한 일을 할 수 있는 가장 약한 Advice를 사용하라**고 권장한다. 예를 들어 단순히 로그만 남기면 된다면 `@Before`로 충분하다. 이렇게 하면 `proceed()` 호출을 깜빡하는 실수도 원천적으로 막을 수 있다.

Aspect : Pointcut과 Advice를 하나로 묶은 모듈

지금까지 나온 개념을 한곳에 모은 것이 **Aspect**다.

- **Pointcut** : 어디에 적용할지
- **Advice** : 무엇을, 언제 할지

Aspect는 이 둘을 묶어 하나의 공통 관심사(예 : 제조 시간 측정)를 모듈화한 것이다. Spring에서는 `@Aspect`가 붙은 클래스가 Aspect가 된다.

```
@Aspect
@Component
```

```
public class TimeAspect {

    @Pointcut("execution(* com.example.food..*.make*
(..))")
    public void makeFood() {} // Pointcut

    @Around("makeFood()")     // Advice
    public Object measureTime(ProceedingJoinPoint
joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long end = System.currentTimeMillis();


System.out.println(joinPoint.getSignature().getName() +
" : " + (end - start) + "ms");
        return result;
    }
}
```

`@Aspect`만 붙여서는 Bean으로 등록되지 않는다. 그래서 `@Component`를 함께 붙이거나 직접 Bean으로 등록해야 Spring이 Aspect로 인식한다.

Introduction : 기존 객체에 새로운 타입을 추가하는 기능

Introduction은 기존 클래스의 코드를 수정하지 않고, AOP 프록시에 새로운 인터페이스와 그 구현을 추가하는 기능이다.

즉, Advice가 기존 메소드 실행 전후에 부가 로직을 끼워 넣는 것이라면, Introduction은 기존 객체가 원래 없던 기능까지 가진 것처럼 만드는 것이다.

```
@Component
public class CookieService {
    public void makeCookie() { ... }
    // ...
}
```

갑자기 모든 음식 서비스에 `something()` 기능을 추가하고 싶다면 어떻게 해야 할까?

```
public interface Traceable {
    void something();
}
```

하지만 기존 클래스들에 하나하나 `implements Traceable`을 붙이는 것은 번거롭다. 클래스가 많을수록, 혹은 수정할 수 없는 클래스라면 더더욱 그렇다. 이때 Introduction을 활용할 수 있다.

먼저 새 인터페이스의 기본 구현체를 만든다.

```
public class DefaultTraceable implements Traceable {

    @Override
    public void something() {
        // ...
    }
}
```

그리고 Aspect에서 `@DeclareParents`로 어떤 타입에 이 인터페이스를 붙일지 선언한다.

```
@Aspect
@Component
public class TraceIntroductionAspect {
```

```
    @DeclareParents(
        value = "com.example.food..*Service+",
        defaultImpl = DefaultTraceable.class
    )
    public static Traceable traceable;
}
```

- `value` : 인터페이스를 추가할 대상 타입이다. `com.example.food` 하위의 `Service`로 끝나는 모든 타입이 대상이며, 끝의 `+`는 **해당 타입의 하위 타입까지 포함**한다는 뜻이다.
- `defaultImpl` : 추가된 인터페이스 메소드가 호출될 때 실제로 동작할 구현 클래스다.
- 필드 타입(`Traceable`) : 추가할 인터페이스를 의미한다.

이렇게 하면 Spring이 만든 프록시는 개념적으로 `class CookieServiceProxy extends CookieService implements Traceable { ... }` 와 비슷한 역할을 한다.

```
CookieService cookieService =
applicationContext.getBean(CookieService.class);

Traceable traceable = (Traceable) cookieService; // 프록시이므로 형변환이 가능하다
traceable.something();                           // DefaultTraceable의 구현이 실행된다
```

주의할 점은 원본 `CookieService`가 실제로 `Traceable`을 구현하게 된 것은 아니라는 것이다. 즉, 새로운 기능은 원본 클래스가 아니라 **프록시에 붙어 있다.**

Advice와 무엇이 다른가?

Advice는 기존 메소드에 개입한다. 즉, 기존 메소드 실행에 부가 기능을 넣는 것이다. Introduction은 원래 없던 메소드 자체를 제공할 수 있다.

> Advice는 **기존 메소드 실행에 기능을 추가**한 것이며, Introduction은 **기존 객체의 타입과 기능 자체를 확장**한 것이다.

Target Object : AOP가 적용되는 원본 객체

Target Object는 AOP가 적용되는 실제 원본 객체이다. 하나 이상의 Aspect로부터 Advice를 받는다는 의미로 **Advised Object**라고도 부른다.

```
@Service
public class CookieService {
    public void makeCookie() {
        // ...
    }
}
```

여기서 Spring이 실제로 만든 `CookieService` 인스턴스가 Target Object이다. 그런데 AOP가 적용되면 Spring은 이 객체를 그대로 외부에 주입하지 않고, 보통 그 앞에 Proxy를 하나 만들어서 대신 주입한다.

```
외부 호출 → AOP Proxy → Advice 실행 → Target Object → makeCookie()
```

AOP Proxy : Target Object를 감싸는 대리 객체

AOP Proxy는 Aspect를 적용하기 위해 Spring AOP가 만드는 대리 객체다. 호출자는 Target Object가 아니라 이 프록시를 주입받아 사용하고,

프록시가 Advice를 실행한 뒤 Target Object에게 실제 작업을 넘긴다.

Spring AOP는 두 가지 방식으로 프록시를 만든다.

방식	사용 조건	특징
JDK 동적 프록시	Target이 인터페이스를 구현한 경우	인터페이스를 기반으로 생성, 인터페이스에 선언된 메소드만 가로챌 수 있다
CGLIB 프록시	인터페이스가 없거나, 클래스 기반 프록시를 강제한 경우	Target 클래스를 **상속**해서 생성한다

Spring Framework 자체는 인터페이스가 있으면 JDK 동적 프록시를 기본으로 사용하지만, **Spring Boot 2.0부터는 CGLIB가 기본값**이다. (`spring.aop.proxy-target-class=true`)

CGLIB는 상속으로 프록시를 만들기 때문에 `final` 클래스는 프록시를 만들 수 없고, `final`·`private` 메소드는 오버라이드할 수 없어 Advice가 적용되지 않는다. 앞서 Join point에서 언급한 제약이 여기서 나온다.

자기 호출(Self-invocation) 문제

```
@Service
public class CookieService {

    public void makeCookieSet() {
        makeCookie(); // this.makeCookie() → 프록시를 거치지 않는다
    }

    public void makeCookie() { ... }
}
```

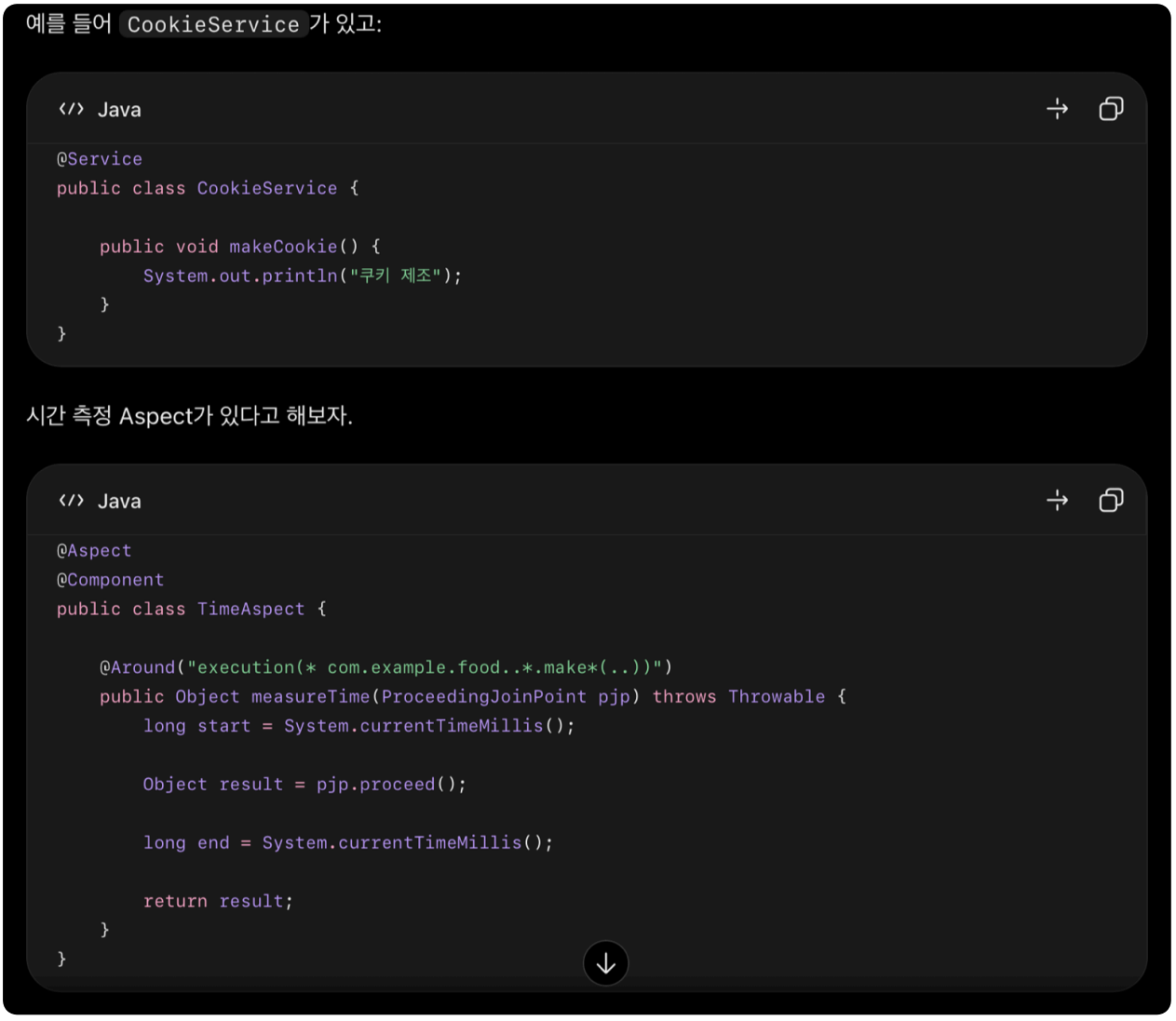
외부에서 `makeCookieSet()`을 호출하면 프록시를 거치므로 `makeCookieSet()`에는 Advice가 적용된다. 하지만 그 안에서 `makeCookie()`를 호출하는 순간, 프록시가 아닌 `this`(Target

Object)를 직접 호출하기 때문에 `makeCookie()`에는 Advice가 적용되지 않는다. `@Transactional`이 같은 클래스 내부 호출에서 동작하지 않는 대표적인 이유도 바로 이것이다.

Weaving : Aspect를 Target Object에 연결하는 과정

Weaving은 Aspect를 Target Object에 실제로 적용해서 동작하도록 만드는 과정이다.

- **Target Object** : 실제 원본 객체
- **Aspect** = Pointcut + Advice
- **Weaving** : 이 Aspect를 Target Object에 연결해서 동작하게 만드는 과정



아직까지는 `CookieService`와 `TimeAspect` 두 개가 따로 존재한다. Spring이 둘을 연결해서 실제 호출이 **CookieService 호출 → TimeAspect의 Advice 실행 → makeCookie() 실행** 순서로 이어지도록 만드는 과정이 Weaving이다. Spring AOP에서는 프록시를 만들어서 Weaving한다.

```
기존       : 호출자 → CookieService
AOP 적용   : 호출자 → CookieService Proxy → Advice → CookieService Target
```

즉, **Weaving**은 Target Object를 프록시로 감싸서 Aspect의 Advice가 호출 과정에 참여하도록 만드는 것이다. Spring에서는 Bean이 생성될 때 자동 프록시 생성기(`BeanPostProcessor`의 일종)가 Pointcut 조건에 맞는 Bean을 찾아 원본 대신 프록시를 컨테이너에 등록하는 방식으로 이 과정이 일어난다.

그리고 Weaving은 언제 적용하는지에 따라 3가지로 나뉜다.

방식	적용 시점	대표 기술
Compile-time weaving	컴파일할 때 Aspect 코드를 바이트코드에 직접 삽입한다	AspectJ 컴파일러(ajc)
Load-time weaving	클래스 로더가 클래스를 JVM에 올릴 때 바이트코드를 조작한다	AspectJ LTW (Java Agent)
Runtime weaving	실행 중 프록시 객체를 생성해서 연결한다	**Spring AOP**

Spring AOP는 **Runtime weaving**을 사용한다. 바이트코드를 직접 바꾸는 것이 아니라 프록시를 앞에 세우는 방식이라 별도의 컴파일러나 Java Agent 없이 쓸 수 있다는 장점이 있다. 대신 프록시가 가로챌 수 있는 메소드 실행만 Join point가 되는 것이 한계다. 처음에 Spring AOP의 Join point가 메소드 실행 하나뿐이라고 했던 이유가 바로 여기에 있다.

한눈에 정리

용어	한 줄 정리	예제에서는
Join point	AOP가 끼어들 수 있는 후보 지점	`makeCookie()` 등의 메소드 실행
Pointcut	후보 중 실제 대상을 고르는 조건식	`execution(* com.example.food..*.make*(..))`
Advice	선택된 지점에서 무엇을, 언제 할지	`@Around measureTime()`
Aspect	Pointcut + Advice를 묶은 모듈	`TimeAspect`
Introduction	프록시에 새 인터페이스와 구현을 추가	`Traceable` + `DefaultTraceable`
Target Object	AOP가 적용되는 원본 객체	`CookieService` 인스턴스
AOP Proxy	Target을 감싸 Advice를 실행하는 대리 객체	`CookieService` 프록시
Weaving	Aspect를 Target에 연결하는 과정	Spring AOP의 Runtime weaving